Енкапсулация.
Абстракция.
Работа с потоци.

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

От какво ще се състои настоящата лекция?

Енкапсулация. Модификатори за достъп. Константни класове и член-функции. Абстракция. Mutable. Потоци и четене и писане от файл.

Какво е енкапсулацията?

- Скриване на вътрешното състояние (data hiding).
- Предоставяне на контролирани методи за достъп (get/set).
- Предпазва от неконсистентни промени.
- Съществуват public, protected и private полета. В момента не сме учили наследяване и private и protected без наследяване се държат еднакво.
- Енкапсулацията повишава надеждността, четимостта и поддръжката на кода, тъй като промените във вътрешната реализация не засягат външния интерфейс.
- Енкапсулацията води до абстракция – позволява да мислим за обекта чрез неговото поведение, без да знаем детайлите на реализацията.

```cpp
class Student {
public:
    void print() const {
        // можем да достъпваме всичко вътре
        std::cout << age_ << grade_ << std::endl;
    }

protected:
    int age_ = 0;

private:
    double grade_ = 0.0;
};

int main() {
    Student s;

    // s.age_ = 20;    // ❌ грешка
    // s.grade_ = 5.5; // ❌ грешка
}
```

```cpp
class Car {
public:
    void start() {
        if (!engineRunning_) {
            startEngine();   // private метод
            engineRunning_ = true;
        }
    }

    void accelerate() {
        if (engineRunning_) {
            injectFuel();    // private метод
            std::cout << "Car is accelerating\n";
        }
    }
    void stop() {
        if (engineRunning_) {
            stopEngine();    // private метод
            engineRunning_ = false;
        }
    }
private:
    bool engineRunning_ = false;
    // Вътрешна логика — скрита от потребителя
    void startEngine() {
        std::cout << "Engine started\n";
    }
    void stopEngine() {
        std::cout << "Engine stopped\n";
    }
    void injectFuel() {
        std::cout << "Fuel injected\n";
    }
};
```

Защо енкапсулацията е важна?

```cpp
class Temperature {
private:
    double celsius_; // internal representation
public:
    // Setter: задава температура във Фаренхайт
    void setFahrenheit(double f) {
        celsius_ = (f - 32.0) * 5.0 / 9.0;
    }
    // Getter: връща температура във Фаренхайт
    double getFahrenheit() const {
        return celsius_ * 9.0 / 5.0 + 32.0;
    }
    // Опционално: getter за Celsius
    double getCelsius() const {
        return celsius_;
    }
};
```

```cpp
#include <iostream>
#include <cmath>
class Point {
private:
    double r_ = 0.0;    // радиус
    double phi_ = 0.0;  // ъгъл в радиани
public:
    // Конструктор чрез декартови координати
    Point(double x = 0.0, double y = 0.0) {
        setXY(x, y);
    }
    // Setter: задава координати чрез x, y
    void setXY(double x, double y) {
        r_ = std::sqrt(x * x + y * y);
        phi_ = std::atan2(y, x);
    }
    // Getter: връща x координата
    double x() const {
        return r_ * std::cos(phi_);
    }
    // Getter: връща y координата
    double y() const {
        return r_ * std::sin(phi_);
    }
    // Принтира координатите
    void print() const {
        std::cout << "(" << x() << ", " << y() <<
")\n};
};
```

Сравнение с езика Java

C++

```cpp
class Temperature {
private:
    double celsius_; // internal representation
public:
    // Setter: задава температура във Фаренхайт
    void setFahrenheit(double f) {
        celsius_ = (f - 32.0) * 5.0 / 9.0;
    }
    // Getter: връща температура във Фаренхайт
    double getFahrenheit() const {
        return celsius_ * 9.0 / 5.0 + 32.0;
    }
    // Опционално: getter за Celsius
    double getCelsius() const {
        return celsius_;
    }
};
```

Java

```java
public class Temperature {
    // internal representation
    private double celsius;

    // Setter: задава температура във Фаренхайт
    public void setFahrenheit(double f) {
        this.celsius = (f - 32.0) * 5.0 / 9.0;
    }

    // Getter: връща температура във Фаренхайт
    public double getFahrenheit() {
        return this.celsius * 9.0 / 5.0 + 32.0;
    }

    // Опционално: getter за Celsius
    public double getCelsius() {
        return this.celsius;
    }
}
```

Абстракция

- Скриване на сложността → по-лесен и безопасен код
- Подобрена четимост и поддръжка
- Позволява работа с високо ниво на концепции
- Улеснява повторна употреба на код
- Позволява да се работи с обекти на ниво какво правят, а не как точно го правят.
- Помага за по-добра модулност и разделяне на отговорностите, тъй като всяка част от системата гледа само своя „абстрактен“ интерфейс.

```cpp
class Car {
public:
    void start() {
        checkEngine();
        std::cout << "Колата стартира\n";
    }
    void stop() {
        std::cout << "Колата спира\n";
    }
    void refuel(int liters) {
        fuel += liters;
        std::cout << "Заредени " << liters << " литра гориво\n";
    }
private:
    int fuel = 0;
    // Детайлът се скрива от потребителя
    void checkEngine() {
        if(fuel <= 0) {
            std::cout << "Грешка: Няма гориво!\n";
        }
    }
};
```

Пример за абстракция

```cpp
class Student {
private:
    std::string name_;
    int age_;
    double grade_;

    // Internal helper — hidden from outside
    bool isValidGrade(double g) const {
        return g >= 2.0 && g <= 6.0;
    }

public:
    // Public interface — getters and setters
    std::string getName() const { return name_; }
    int getAge() const { return age_; }
    double getGrade() const { return grade_; }

    void setGrade(double g) {
      // note it's not a trivial setter
        if (!isValidGrade(g)) {
            if (g < 2.0) g = 2.0;
            if (g > 6.0) g = 6.0;
        }
        grade_ = g;
    }

    // Public method showing behavior, hides internal logic
    void promote() {
        if (grade_ < 6.0) {
            grade_ += 0.5;
            if (grade_ > 6.0) grade_ = 6.0;
        }
        std::cout << name_ << " promoted to grade " << grade_ << std::endl;
    }

    void printInfo() const {
        std::cout << "Name: " << name_
                  << ", Age: " << age_
                  << ", Grade: " << grade_
                  << std::endl;
    }
};
```

Константността в езика C++

Константността в C++ е механизъм за гарантиране, че определени данни няма да бъдат променяни.

Защо е полезна:

1. Предотвратява случайна модификация на данните.
2. Подобрява разбирането на кода.
3. Позволява оптимизации от компилатора.
4. Безопастност при четенето на данни
5. По подразбиране всички променливи в C++ могат да бъдат променяни, докато в езици като Rust е обратното

```cpp
//C++
class Point {
    double x, y;
public:
    double getX() const { return x; } // //const function
    double getY() const { return y; }
    void setX(double val) { x = val; }
};
//Rust
struct Point {
    x: f64,
    y: f64,
}

impl Point {
    fn get_x(&self) -> f64 {
        self.x
    }

    fn get_y(&self) -> f64 {
        self.y
    }

    fn set_x(&mut self, val: f64) {
        self.x = val;
    }
}
```

Константни обекти

1. Константен обект може да извиква само константни функции.
2. Позволява работа с обекти, които не трябва да се променят.
3. Защитава обекта от случайни промени.
4. Ясно показва на програмиста кои функции не модифицират състоянието.

```
C++:
const Point p;
// p.setX(5); // ❌ грешка: не може да се извика неконстантна функция
double x = p.getX(); // ✅ позволено

Rust:
    let p = Point { x: 1.0, y: 2.0 };   // immutable променлива (аналог на const Point)
    // p.set_x(5.0);     // ❌ грешка: не може да заемем &mut self от immutable променлива

    let x = p.get_x();   // ✅ позволено (само &self)
    println!("{}", x);

    let mut q = Point { x: 1.0, y: 2.0 }; // mutable променлива
```

Константни класове

1. Предимство: След създаване на обекта, стойностите му не могат да се променят.
2. Подходящ за непроменяеми структури, например координати на точки, цветове, конфигурации.
3. Използва се често при многонишковото програмиране
4. Не съществуват като понятие в езика C++, но могат да бъдат направени с конструкциите на езика

```
class ImmutablePoint {
    double x, y;
public:
    double getX() const { return x; }
    double getY() const { return y; }
};
```

Ключова дума mutable

дефиниция

Позволява модификация на членове въпреки const функцията.

Полезно за:

1)Кеширане на резултати
2)Броене на достъпи
3)Логове

Пример

```
class Counter {
   mutable int count = 0;
public:
   void increment() const { count++; }
   int getCount() const { return
count; }
};

int main(){
        const Counter c;
        c.increment(); // ✅ работи,
защото count е mutable
        std::cout << c.getCount();
        return 0;
}
```

Копие

Без mutable компилаторът би върнал грешка, че не може да се променя елемент в константа функция.

Позволява да се пазят вътрешни данни, които не променят семантиката на обекта.

Обобщение на наученото до момента

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string name_;
    int age_;
    mutable int accessCount_ = 0; // mutable за броене на достъпи до
public:
     //Generate constructor!
    // Getter-и (константни) - абстракция и const correctness
    std::string getName() const {
        accessCount_++; // брои всяко извикване
        return name_;
    }
    int getAge() const {
        accessCount_++;
        return age_;
    }
    // Setter-и (енткапсулация)
    void setName(const std::string& name) { name_ = name; }
    void setAge(int age) { age_ = age; }

    // Метод за извеждане (const)
    void print() const {
        accessCount_++;
        std::cout << "Name: " << name_  << ", Age: " << age_
    }
    // Вътрешна статистика (mutable)
    int getAccessCount() const { return accessCount_; }
};
int main() {
    const Student s("Ivan", 20, 5.50);
    s.print();              // ✅ const функция
    std::cout << s.getName() << std::endl; // ✅ const функция
    std::cout << "Access count: " << s.getAccessCount() << std::endl;
 }
```

Потоци. Четене и писане от файл.

Дефиниция

Потоците са абстракция за вход и изход (I/O) на данни и са последователност от данни, която може да бъде прочетена или записана.

Позволяват работа с файлове, клавиатура и други устройства.

Пример

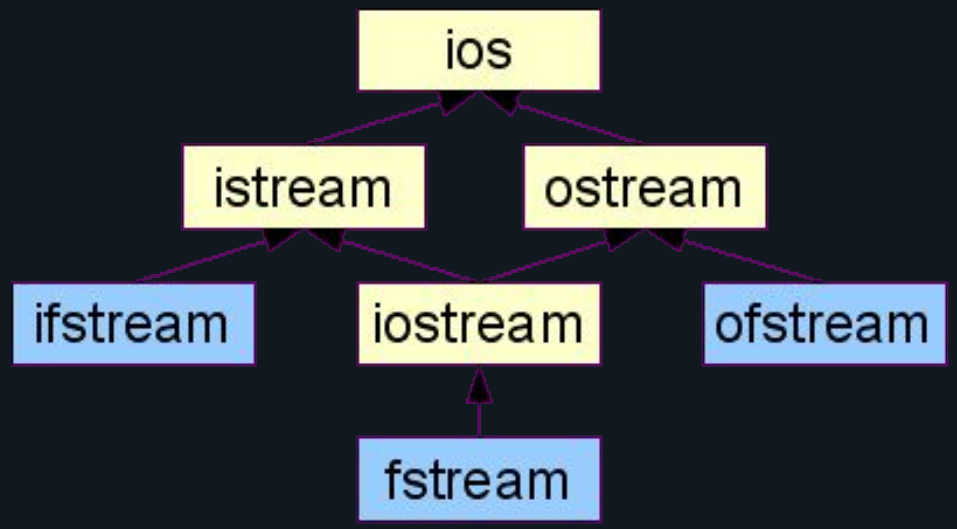
```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;
    cout << "You are " << age << "
years old." << endl;
    return 0;
}
```

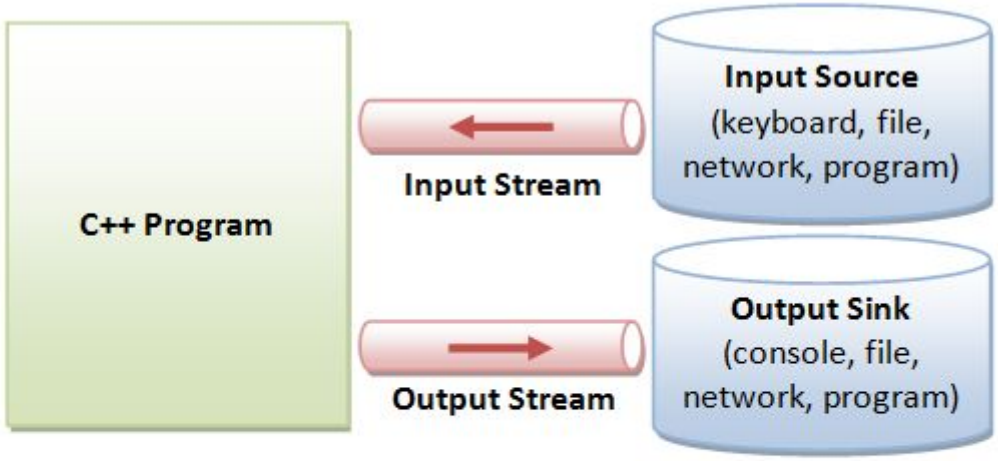
Библиотеки

```
#include <iostream>   // std::cin, std::cout,
std::cerr
#include <fstream>    // std::ifstream,
std::ofstream, std::fstream
#include <sstream>    // std::stringstream,
std::istringstream, std::ostringstream ->
```
текстови потоци

Йерархия на класовете



Визуално обяснение



Internal Data Formats:

- Text: `char`, `wchar_t`
- `int`, `float`, `double`, etc.

External Data Formats:

- Text in various encodings (US-ASCII, ISO-8859-1, UCS-2, UTF-8, UTF-16, UTF-16BE, UTF16-LE, etc.)
- Binary (raw bytes)

Потоци - примери

```
#include <iostream>
using namespace std;

int main() {
    string name = "Todor";
    cout << "Hello, " << name << "!" << endl;
    return 0;
}

//endl – нов ред и изчистване на буфера.
//Може да се използва и \n, но endl гарантира
записване в изхода.
```

Пример за четене от файл

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
    ifstream inputFile("data.txt");
    string line;
    if (inputFile.is_open()) {
        while (getline(inputFile, line)) {
            cout << line << endl;
        }
        inputFile.close();
    } else {
        cout << "Unable to open file." <<
endl}
    return 0;
}
```

Потоци - примери

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    fstream file("data.txt", ios::in | ios::out | ios::app);
    file << "Appending this line." << endl;
    file.close();
    return 0;
}

//fstream може едновременно да чете и пише.
//ios::in – четене, ios::out – запис, ios::app – //добавяне в края на
файла.
```

Пример за запис във файл

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream outputFile("output.txt");

    if (outputFile.is_open()) {
        outputFile << "Hello, world!" << endl;
        outputFile << "C++ file I/O example." <<
endl;   outputFile.close();
    } else {
        cout << "Unable to open file." << endl;
    }
    return 0;
}
```

Потоци - примери

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream out("numbers.txt");
    out << 10 << " " << 20 << " " << 30 << endl;
    out.close();

    ifstream in("numbers.txt");
    int a, b, c;
    in >> a >> b >> c;
    cout << a << ", " << b << ", " << c << endl;
    in.close();

    return 0;
}

// четене и запис във файл
```

Добри практики:

1)Винаги проверявайте дали файлът е успешно отворен.

2)Затваряйте файловете след работа (close() или RAII чрез fstream).

3)Използвайте буфери и flush() при нужда от незабавен запис.

4)Избягвайте хардкоднати пътища – използвайте относителни или конфигурационни файлове.

5)Потоците са универсален инструмент за работа с вход и изход в C++.

Обобщен пример с класа Student

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string name_;
    int facultyNumber_;
    double grade_;

public:
    Student(std::string name, int fn, double grade)
        : name_(name), facultyNumber_(fn), grade_(grade) {}

    // Декларации на приятелски оператори
    friend std::istream& operator>>(std::istream& in, Student& s);
    friend std::ostream& operator<<(std::ostream& out, const Student&
};

std::istream& operator>>(std::istream& in, Student& s) {
    // Формат:
    // име факултетен_номер оценка
    in >> s.name_ >> s.facultyNumber_ >> s.grade_;
    return in;
}

std::ostream& operator<<(std::ostream& out, const Student& s) {
    out << "Name: " << s.name_
        << ", FN: " << s.facultyNumber_
        << ", Grade: " << s.grade_;
    return out;
}
```

Въпроси?

Благодаря за вниманието!

допълнителни материали: learncpp.com, глава 28